Day 69: The Week 10 Build — a Seq2Seq Encoder-Decoder From the Exercise Parts

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 10, Day 7 | Complete study guide

Watch: Sequence-to-Sequence (seq2seq) Encoder-Decoder Neural Networks, Clearly Explained!!! (StatQuest) | Paper Review: Sequence to Sequence Learning with Neural Networks (Aladdin Persson)

Script: `day69_seq2seq_encoder_decoder.py` · every number in this guide is copied by program from `day69_run_output.txt`, the real output of that script.

Week 10's last day assembles this week's exercise scripts into one model: an **encoder** LSTM reads a sequence of digits and hands only its final state to a **decoder** LSTM, which writes an output sequence one digit at a time. Trained to COPY or to REVERSE sequences of 3 to 8 digits, it gets 0.9917 and 0.9950 of the test sequences exactly right. Removing the encoder's state drops it to near guessing, so everything really does pass through those 64 numbers. Errors grow towards the end of the longest sequences, which is where Day 70 begins.

0. Where today fits: the parts

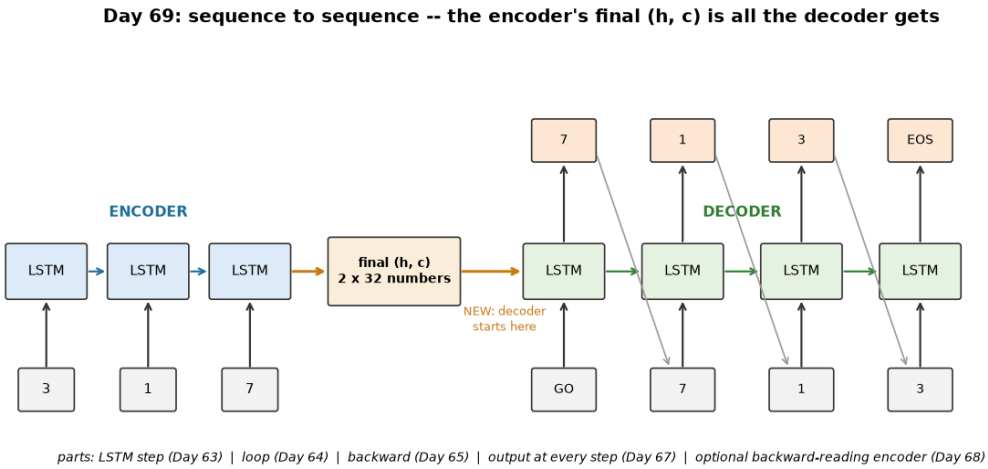
DEFINITION — start here, no prior knowledge assumed

A **sequence-to-sequence** (seq2seq) model maps an input sequence to an output sequence that may have a different length (Sutskever et al., 2014). The **encoder** reads the whole input; its final state, sometimes called the *thought vector*, is the decoder's starting state. The **decoder** then writes the output one token at a time, each step reading the token before it.

REAL-WORLD SCENARIO

An interpreter listens to a whole sentence, keeps its meaning in mind, then speaks the translation word by word. Everything the second language needs must survive in the interpreter's memory between listening and speaking.

SYSTEM DIAGRAM



REVERSE of 3 1 7: the encoder reads the digits; its final (h, c) starts the decoder, which reads GO and then its own previous output, and writes 7 1 3 EOS.

Like Day 62 for Week 9, the model is assembled from this week's short exercise scripts. Lines that had to change are marked CHANGED in the script; lines that exist only because of today's topic are marked NEW.

ACTUAL OUTPUT

```
from              what is reused                        changed today?
Day 63 exercise   sigmoid, lstm_step_forward            no -- verbatim
Day 64 exercise   lstm_forward (the loop)               no -- verbatim (day65's form)
Day 65 exercise   lstm_backward                         3 lines: gradient in at c, out at h0, c0
Day 66 exercise   gru_step_forward                      not used
Day 67 exercise   a softmax and loss at every step      moved into its own function
Day 68 exercise   a second LSTM reading backwards       final states only
```

How to read each section

Step	What it gives you
DEFINITION	What the idea is, in plain words. Start here.
REAL-WORLD SCENARIO	An everyday situation where the same idea shows up.
MATHEMATICS	The formula, when the section introduces one.
SPECIFICATION	The exact setup of the experiment.
Why this test	The question the experiment answers.
IMPLEMENTATION	The real code from the script.
ACTUAL OUTPUT	What the script really printed, copied unchanged.
What it means	How to read the numbers, and what they do and do not show.
Why choose this? / Why NOT this?	When the idea is the right tool, and what it costs.

1. The task and the assembled model

DEFINITION — start here, no prior knowledge assumed

Ten tokens: digits 1–8, **GO** (9, “start writing”) and **EOS** (0, “end of sequence”). The decoder *reads* GO followed by the target, and must *write* the target followed by EOS: at every step it predicts the next token. During training it is given the true previous token (**teacher forcing**); in use it reads its own previous output (**greedy decoding**).

REAL-WORLD SCENARIO

Dictation: the student hears a sentence, then writes it out, each word chosen knowing the words already written. In class the teacher corrects each word immediately (teacher forcing); in the exam nobody does.

MATHEMATICS

$$\textit{encoder: } (h_t, c_t) = LSTM_{enc}(x_t, h_{t-1}, c_{t-1}), \textit{ from zeros; keep } (h_n, c_n)$$

$$\textit{decoder: } (s_0, m_0) = (h_n, c_n); \quad (s_j, m_j) = LSTM_{dec}(y_{j-1}, s_{j-1}, m_{j-1}), \quad y_0 = GO$$

$$p(y_j) = softmax(s_j W_{hy} + b_y); \quad L = \textit{mean cross-entropy over every output step}$$

SPECIFICATION

Model	Hidden 32 for encoder and decoder; one-hot inputs; forget bias 1.
Worked example	Two REVERSE sequences of 5 digits, with what the decoder reads and writes.

Why this test

Seeing the decoder's input and target side by side shows the one-step shift that teacher forcing relies on.

IMPLEMENTATION

```
def make_batch(rng, m, n, task):
    src = rng.randint(1, 9, size=(m, n))
    tgt = src if task == "copy" else src[:, ::-1]
    dec_in = np.concatenate([np.full((m, 1), GO), tgt], axis=1)
    dec_out = np.concatenate([tgt, np.full((m, 1), EOS)], axis=1)
    return np.eye(V)[src], np.eye(V)[dec_in], dec_out, src
```

ACTUAL OUTPUT

```
worked example (REVERSE, 5 digits; EOS = 0, GO = 9):
  source [5, 8, 6, 1, 4]  decoder reads [9, 4, 1, 6, 8, 5]  decoder must write [4, 1, 6, 8, 5, 0]
  source [4, 4, 8, 2, 4]  decoder reads [9, 4, 2, 8, 4, 4]  decoder must write [4, 2, 8, 4, 4, 0]
left-to-right encoder: Wef (42, 128), bef (128,), Wd (42, 128), bd (128,), Why (32, 10), by (10,);
    total 11338
bidirectional encoder: Wef (26, 64), bef (64,), Web (26, 64), beb (64,), Wd (42, 128), bd (128,),
    Why (32, 10), by (10,); total 9290
Actual output from this lesson's run.
```

What it means

The decoder input is the target shifted right by one, with GO in front; the target ends with EOS, so the model learns when to stop. The whole model has 11338 weights with the left-to-right encoder and 9290 with the bidirectional one (two encoder LSTMs of 16 units).

WHY CHOOSE THIS?

A toy task with a known correct answer: any error is the model's, and accuracy can be exact.

WHY NOT THIS? (TRADEOFFS)

Copying and reversing are much easier than translation; this tests the mechanism, not language.

2. A gradient check through the decoder and into the encoder

DEFINITION — start here, no prior knowledge assumed

The one new line today: the decoder starts from the encoder's final (h, c). So the decoder's backward pass must also return the gradient at its starting states, and the encoder's backward pass starts from that. The encoder's weights get gradient *only* this way.

REAL-WORLD SCENARIO

A relay where the only thing passed on is the baton: if the second runner's time depends on how the baton was handed over, the first runner's coach needs that feedback too.

MATHEMATICS

dL/dh_n = (decoder's dL/ds_0), dL/dc_n = (decoder's dL/dm_0) → *the encoder's backward pass starts here, with no other gradient*

SPECIFICATION

Data	Hidden 4 (bidirectional encoder 2 + 2), 3 REVERSE sequences of 5 digits.
Pass rule	Day 62's, for every entry of every parameter.

Why this test

If the gradient handed from decoder to encoder were wrong, only the encoder's rows would fail, which is exactly what this check would show.

IMPLEMENTATION

```
def lstm_backward(dHs, dh_last, dc_last, X, h0, c0, W, Hs, Cs, gates):
    m, T, D = X.shape
    dW, db = np.zeros_like(W), np.zeros(W.shape[1])
    dh, dc = dh_last.copy(), dc_last.copy()
    for t in reversed(range(T)):
        dh = dh + dHs[:, t, :]
        f, i, g, o = gates[t]
        h_prev = Hs[:, t - 1, :] if t > 0 else h0
        c_prev = Cs[:, t - 1, :] if t > 0 else c0
        tanh_c = np.tanh(Cs[:, t, :])
        do = dh * tanh_c
        dc = dc + dh * o * (1 - tanh_c ** 2)
        da = np.concatenate([dc * c_prev * f * (1 - f),
                             dc * g * i * (1 - i),
                             dc * i * (1 - g ** 2),
                             do * o * (1 - o)], axis=1)
        z = np.concatenate([X[:, t, :], h_prev], axis=1)
        dW += z.T @ da
        db += da.sum(axis=0)
        dh = (da @ W.T)[:, D:]
        dc = dc * f
    return dW, db, dh, dc
```

The three CHANGED lines: gradient may arrive at the last c as well as the last h; and the function returns the gradients at h0 and c0.

IMPLEMENTATION

```
g["Wd"], g["bd"], dh0, dc0 = lstm_backward(dHs, z, z, Xd, h0, c0, self.P["Wd"], Hd, Cd, gd)
He = self.P["Wef"].shape[1] // 4
for j, (d, X, Hs, Cs, gates) in enumerate(caches):
    part = slice(j * He, (j + 1) * He)
    ze = np.zeros((X.shape[0], He))
    g["We" + d], g["be" + d], _, _ = lstm_backward(np.zeros_like(Hs), dh0[:, part], dc0[:, part],
                                                   X, ze, ze, self.P["We" + d], Hs, Cs, gates)
```

ACTUAL OUTPUT

```
encoder         parameter  entries  max relative error  max |difference|   pass
left-to-right         Wef      224            5.76e-06          3.95e-11   True
left-to-right         bef       16            1.49e-05          3.03e-11   True
left-to-right          Wd      224            3.77e-06          3.63e-11   True
left-to-right          bd       16            1.49e-08          2.08e-11   True
left-to-right         Why       40            1.01e-07          3.63e-11   True
left-to-right          by       10            1.42e-09          2.23e-11   True
bidirectional         Wef       96            5.71e-05          3.22e-11   True
bidirectional         bef        8            4.20e-07          2.09e-11   True
bidirectional         Web       96            1.78e-04          4.06e-11   True
bidirectional         beb        8            1.60e-07          2.48e-11   True
bidirectional          Wd      224            1.46e-06          4.00e-11   True
bidirectional          bd       16            1.70e-08          3.04e-11   True
bidirectional         Why       40            6.02e-08          4.20e-11   True
bidirectional          by       10            3.90e-10          2.27e-11   True
every entry passes: left-to-right encoder True, bidirectional encoder True
Actual output from this lesson's run.
```

What it means

Every entry passes, for both encoders, including the encoder weights that receive gradient only through the decoder's initial state (largest relative error 1.78e-04 on the backward encoder's W, where the absolute difference is 4.06e-11).

WHY CHOOSE THIS?

Confirms the only new piece of code before any training result is trusted.

WHY NOT THIS? (TRADEOFFS)

A tiny model; the trained model is larger but uses the same code.

3. Training: COPY and REVERSE

DEFINITION — start here, no prior knowledge assumed

Exact match counts a sequence as right only if every digit and the final EOS are right. **Token accuracy** counts individual output tokens. Both are measured with greedy decoding, the way the model would be used.

REAL-WORLD SCENARIO

Marking a dictation: exact match is “no mistakes at all”, token accuracy is the share of words spelled right.

SPECIFICATION

Training	Plain SGD, lr 0.5, clip 1.0; batches of 32 sequences of one random length 3–8; 20,000 steps; seeds 0, 1, 2.
Test	300 fixed sequences of every length 3–8 per task, the same for every model.

Why this test

Can a single state vector carry a whole sequence, and does it matter in which order it is written back?

IMPLEMENTATION

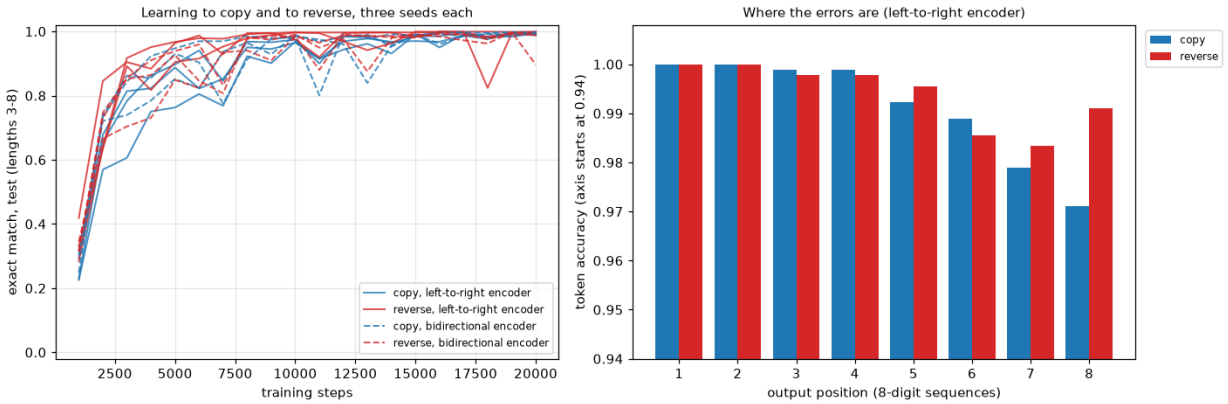
```
def generate(self, Xs, n_steps, zero_state=False):
    h, c, _ = self.encode(Xs)
    if zero_state:
        h, c = np.zeros_like(h), np.zeros_like(c)
    token = np.full(len(Xs), GO)
    out = []
    for _ in range(n_steps):
        h, c, _ = lstm_step_forward(np.eye(V)[token], h, c, self.P["Wd"], self.P["bd"])
        token = (h @ self.P["Why"] + self.P["by"]).argmax(axis=1)
        out.append(token)
    return np.stack(out, axis=1)
```

ACTUAL OUTPUT

```
task     encoder         seed  final loss  exact match  token acc  seconds
copy     left-to-right      0      0.0093       0.9922     0.9985     64.9
copy     left-to-right      1      0.0081       0.9894     0.9975     72.6
copy     left-to-right      2      0.0105       0.9933     0.9983     72.5
reverse  left-to-right      0      0.0024       0.9994     0.9999     72.9
reverse  left-to-right      1      0.0045       0.9872     0.9969     74.4
reverse  left-to-right      2      0.0051       0.9983     0.9996     77.8
Actual output from this lesson's run.
```

ACTUAL OUTPUT

```
exact match by length (mean of 3 seeds):
task           n=3      n=4      n=5      n=6      n=7      n=8
copy         1.000    1.000    0.999    0.998    0.991    0.962
reverse      1.000    1.000    1.000    1.000    0.992    0.978
Actual output from this lesson's run.
```



Left: exact match on the test sets during training, every seed (solid: left-to-right encoder, dashed: bidirectional). Right: accuracy at each output position for 8-digit sequences.

What it means

Both tasks are learned almost perfectly at these lengths. Every sequence of 3 or 4 digits is right; accuracy falls only at the longest lengths, to 0.962 (copy) and 0.978 (reverse) exact match at 8 digits. Reverse is slightly easier, as Sutskever et al. found when they reversed their source sentences, but here the difference is small.

WHY CHOOSE THIS?

Shows the encoder-decoder mechanism working end to end, on code built entirely from this week's parts.

WHY NOT THIS? (TRADEOFFS)

The longest training sequences are already where errors start; longer ones are Day 70's question.

4. Two controls

DEFINITION — start here, no prior knowledge assumed

A **control** removes one ingredient to prove it was needed. (a) Replace the encoder's final state with zeros at test time: the decoder now knows nothing about the input. (b) Compare teacher-forced accuracy with greedy accuracy: if the model's own mistakes snowballed, greedy would be much worse.

REAL-WORLD SCENARIO

(a) Ask the interpreter to translate a sentence they never heard. (b) Compare a dictation corrected word by word with one marked only at the end.

SPECIFICATION

(a)	The trained models, encoder state set to zeros; guessing a digit is right 1 time in 8.
(b)	Token accuracy with the true previous token as input vs with the model's own previous output.

Why this test

It checks that the information really flows through the encoder's state, and that training-time accuracy is not hiding a problem that appears only in real use.

IMPLEMENTATION

```
def teacher_forced(self, Xs, Xd):
    h0, c0, _ = self.encode(Xs)
    Hd, _, _ = lstm_forward(Xd, h0, c0, self.P["Wd"], self.P["bd"])
    return (Hd @ self.P["Why"] + self.P["by"]).argmax(axis=2)
```

ACTUAL OUTPUT

```
task         normal  encoder state zeroed  teacher-forced   greedy
copy         0.9981                0.1872          0.9990   0.9981
reverse      0.9988                0.1864          0.9994   0.9988
Actual output from this lesson's run.
```

What it means

(a) Without the encoder's state, token accuracy collapses from 0.9981 to 0.1872 (copy): about the level of guessing (1 in 8 per digit); the small excess above 0.125 was not analysed. Everything about the input passes through the 64 numbers of (h, c).

(b) Teacher-forced and greedy accuracy are almost the same (0.9990 against 0.9981 for copy): at these lengths the model rarely makes the first mistake that greedy decoding would then build on.

WHY CHOOSE THIS?

Two cheap tests that make the main result believable.

WHY NOT THIS? (TRADEOFFS)

Exposure to its own mistakes may matter more on harder tasks, where teacher forcing and greedy decoding diverge.

5. Where the errors are

DEFINITION — start here, no prior knowledge assumed

Accuracy at each output position for the 8-digit test sequences. For COPY, output position 1 is the digit the encoder read first, longest before its final state; for REVERSE, it is the digit read last.

REAL-WORLD SCENARIO

Which words of a long dictated sentence go wrong: the ones heard longest ago, or the ones written last?

SPECIFICATION

Data	300 test sequences of 8 digits per task; greedy decoding; mean of three seeds.

Why this test

If the encoder's memory faded with time, COPY's first output positions (read longest ago) should fail first. The test checks that expectation.

IMPLEMENTATION

```
Xs, _, Y, src = test_sets(task)[8]
acc[task] = np.mean([(train_cached(task, False, s)[0].generate(Xs, 9)[:, :8] == Y[:,
    :8]).mean(axis=0)
                     for s in SEEDS], axis=0)
```

ACTUAL OUTPUT

```
output position            1      2      3      4      5      6      7      8
COPY: read at step         1      2      3      4      5      6      7      8
  accuracy             1.000  1.000  0.999  0.999  0.992  0.989  0.979  0.971
```

```
REVERSE: read at step       8      7      6      5      4      3      2      1
  accuracy              1.000  1.000  0.998  0.998  0.996  0.986  0.983  0.991
Actual output from this lesson's run.
```

What it means

The expectation was wrong. For COPY the first positions are perfect (1.000) and the errors are at the end (0.971 at position 8), even though position 8 is the digit the encoder read *most recently*. REVERSE shows the same shape: its weakest positions are near the end of the output (0.983 at position 7). What predicts errors here is how far into its output the decoder is, not how long ago the encoder read the digit.

A likely reason, not tested here: the decoder must carry both the remaining digits and its place in the sequence in its own state, and that gets harder with every step it writes. Day 70 measures what happens when sequences get longer than these.

WHY CHOOSE THIS?

Checks an intuitive explanation instead of assuming it, and finds it does not hold.

WHY NOT THIS? (TRADEOFFS)

Errors are rare at these lengths (a few percent), so the per-position numbers rest on few mistakes.

6. Day 68's bidirectional encoder

DEFINITION — start here, no prior knowledge assumed

Day 68's idea in the encoder: one LSTM reads the input left to right, another right to left, and their final states are joined. To keep the decoder at 32 units, each encoder direction has 16.

REAL-WORLD SCENARIO

Two listeners, one hearing the sentence normally and one hearing it backwards, pooling what they remember.

SPECIFICATION

Runs	Both tasks × both encoders × seeds 0, 1, 2; everything else as in Section 3.

Why this test

A backward-reading encoder ends next to the FIRST input digit, which COPY must write first.

IMPLEMENTATION

```
if self.bi:
    Xr = Xs[:, ::-1]
    Hr, Cr, gr = lstm_forward(Xr, z, z, self.P["Web"], self.P["beb"])
    caches.append(("b", Xr, Hr, Cr, gr))
    h = np.concatenate([h, Hr[:, -1]], axis=1)
    c = np.concatenate([c, Cr[:, -1]], axis=1)
```

ACTUAL OUTPUT

```
task     encoder           seed 0   seed 1   seed 2     mean  best checkpoint  seconds
copy     left-to-right     0.9922   0.9894   0.9933   0.9917           0.9946     70.0
copy     bidirectional     0.9961   0.9983   0.9933   0.9959           0.9967     77.3
reverse  left-to-right     0.9994   0.9872   0.9983   0.9950           0.9993     75.1
reverse  bidirectional     0.9972   0.9906   0.8967   0.9615           0.9957     78.7
Actual output from this lesson's run.
```

What it means

COPY: 0.9959 exact match with the bidirectional encoder against 0.9917, a small gain. REVERSE: 0.9615 against 0.9950, because one seed ended at 0.8967.

That low number is a late dip, not a failure to learn: the best checkpoint column (mean 0.9957) and the learning curves show the run near the others until a drop at the very last checkpoint. Plain SGD at lr 0.5 has such spikes (a left-to-right REVERSE run also dips at 18,000 steps and recovers). Reading only the final step would have called it a failure; this is why the series tracks accuracy during training. A smaller learning rate or keeping the best checkpoint would remove the effect; neither was tried here.

Time per run: 77.3 and 78.7 seconds for the bidirectional encoder against 70.0 and 75.1 for the left-to-right one, in this run. Two small encoder loops instead of one cost a little more, since each step pays roughly the same Python overhead whatever its size. This machine's speed varied between runs of the same script, so only times from the same run should be compared.

WHY CHOOSE THIS?

Reuses Day 68's exercise directly; each direction has half the units, so the total size stays similar.

WHY NOT THIS? (TRADEOFFS)

At lengths where the plain encoder already succeeds there is little room to gain, and one run's late dip decides the REVERSE comparison.

Interview preparation — technical questions

Q: How does the encoder pass information to the decoder in a basic seq2seq model?

A: Only through its final hidden and cell states, which become the decoder's initial states.

Q: What changes in backpropagation when the decoder starts from the encoder's state?

A: The decoder's backward pass must return the gradient at its initial (h, c); the encoder's backward pass starts from it.

Q: What is teacher forcing?

A: Training the decoder with the true previous token as input at every step, instead of its own prediction.

Q: How do you check that the decoder actually uses the encoder?

A: Zero the encoder's state at test time; accuracy should collapse (here to about guessing).

Q: Why does the decoder need GO and EOS tokens?

A: GO gives the first step an input; EOS lets the model decide when the output ends.

Interview preparation — theoretical questions

Q: What is the bottleneck of a basic encoder-decoder?

A: The whole input must fit into one fixed-size state. As inputs get longer, that state must hold more; Day 70 measures the effect.

Q: Why did Sutskever et al. reverse the source sentence?

A: It puts the first source words next to the first target words in time, shortening the dependencies the model must learn. Here reversing was slightly easier than copying.

Q: What is exposure bias?

A: A model trained only with teacher forcing never sees its own mistakes as inputs; at test time one mistake can lead to more. Here teacher-forced and greedy accuracy were almost equal.

Q: Can the decoder be bidirectional?

A: No: it writes one token at a time and cannot read tokens it has not written yet. The encoder can.

Key takeaway and what's next

> **KEY TAKEAWAY**
>
> Two LSTMs built from this week's exercise scripts, joined only by one final state, learn to copy and to reverse digit sequences almost perfectly at lengths 3 to 8, and the zero-state control proves the input really travels through that state. The one new line of code, the gradient handed from decoder to encoder, checks out exactly. The errors that remain sit at the end of the longest outputs, not at the digits read longest ago: an honest surprise, and the starting point for Day 70.

- Encoder reads; its final (h, c) is the decoder's starting state; the decoder writes one token per step.
- New code: lstm_backward also returns dL/dh0 and dL/dc0; the encoder's backward pass starts from them.
- Exact match (lengths 3–8): copy 0.9917, reverse 0.9950.
- Encoder state zeroed: token accuracy about 0.19, near guessing.
- Teacher-forced and greedy accuracy almost equal at these lengths.
- Errors grow with output position, for both tasks.

The week in one paragraph

Day 63 built the LSTM cell and its cell-state route; Day 64 looped it over time; Day 65 backpropagated through it and found the forget bias decides whether it learns; Day 66 built the GRU and found where its tied gate costs it; Day 67 put all three cells on a real task and found the cell matters little when the needed context is short; Day 68 read in both directions (+2 points) and stacked layers (no gain). Today two LSTMs became an encoder and a decoder.

Suggested watch order, if starting from zero

- 1. Sequence-to-Sequence (seq2seq) Encoder-Decoder Neural Networks, Clearly Explained!!! (StatQuest) — the encoder-decoder idea, step by step.
- 2. Paper Review: Sequence to Sequence Learning with Neural Networks (Aladdin Persson) — the original paper: reversed inputs, deep LSTMs, and the results.

Day 70 preview

- **The information bottleneck.** Everything the decoder knows passes through one state of fixed size. Day 70 trains and tests on longer sequences to show where that breaks, which motivates attention (Day 71).